

PARCIAL 2: Teoría de Aprendizaje de Máquina - 2025 - 1

Integrantes:

Felipe Idarraga Quintero

Julian Felipe Gutierrez Ramirez

Samuel Marulanda Salazar

MODELOS DE APRENDIZAJE DE MÁQUINA:

PCA: (Principal Component Analysis).

Es una técnica de reducción de dimensionalidad que busca proyectar datos de alta dimensión a un espacio de menor dimensión preservando la mayor cantidad de Varianza posible. Se utiliza para Compresión, Visualización y eliminación de ruido.

- Perspectiva Generativa:

Se asume que los datos originales están en una matriz: $X \in \mathbb{R}^{N \times P}$, donde:

N : Número de muestras, P : Número de Características (dimensión Original)

En PCA se asume que los datos se generan desde un espacio de baja dimensión:

$$Z \in \mathbb{R}^{N \times M}, \text{ con } M < P$$

Usando una matriz de componentes principales $W \in \mathbb{R}^{P \times M}$, se reconstruye la matriz original como:

$$X = ZW^T$$

Esto implica que

$$Z = XW$$

- Problema de Optimización:

Se busca la matriz W que minimice el error de reconstrucción

$$W^* = \underset{W}{\operatorname{Argmin}} \|X - XWW^T\|_F$$

S.t. $WW^T = I$, las columnas W deben ser ortogonales

- Perspectiva de Varianza (Geométrica):

Otra forma de ver PCA es como el proceso de encontrar una proyección que maximiza la Varianza de los datos proyectados.

Dado que los datos están centrados en la media, su matriz de covarianza es:

$$\Sigma = \frac{1}{N} X^T X$$

La nueva formulación del problema de Optimización es:

$$W^* = \arg \max_W \text{tr}(W^T \Sigma W)$$

$$\text{s.t. } W W^T = I$$

Esto significa que se está buscando los M vectores propios de Σ (matriz de Covarianza) que maximicen la Varianza.

UMAP:

UMAP construye un ponderado entre puntos x_n y $x_{n'}$, usando una probabilidad de Conectividad:

$$p_{nn'} = \exp\left(-\frac{\|x_n - x_{n'}\| - P_n}{\sigma_n}\right)$$

donde: P_n : distancia al vecino más cercano de x_n .

σ_n : Escala adaptativa según el número de Vecinos.

Para evitar que una matriz asimétrica de similitudes, UMAP simetriza:

$$\tilde{p}_{nn'} = p_{nn'} + p_{n'n} - p_{nn'} p_{n'n}, \text{ esto produce un grafo ponderado, no dirigido}$$

En el espacio de baja dimensión (con puntos z_n), se define una probabilidad $q_{nn'}$ usando una función de tipo t-student de cola pesada:

$$q_{nn'} = (1 + a \|z_n - z_{n'}\|^{2b})^{-1}$$

los parámetros a y b controlan la forma de la distribución; generalmente se fijan en 1.

UMAP minimiza la entropía cruzada:

$$C(P; Q) = \sum_{n \neq n'} (-\tilde{p}_{nn'} \log(q_{nn'}) - (1 - \tilde{p}_{nn'}) \log(1 - q_{nn'}))$$

• Naive Bayes Gaussian

Es un clasificador supervisado probabilístico basado en el teorema de Bayes, con una suposición de independencia "naive" entre las características, y una suposición adicional que cada característica X_i sigue una distribución normal dentro de cada clase A .

Formula general (con independencia)

$$P(X, A) = P(X|A)P(A) = \prod_{j=1}^p P(x_j|A) \cdot P(A)$$

• $X = (x_1, x_2, \dots, x_p)$: vector de características

• A : clase

• $P(A)$: probabilidad a priori de la clase (prior)

• $P(x_j|A)$: verosimilitud (likelihood) de la característica j dado que la clase es A

Predicción:

$$\hat{y} = f(X) = \arg \max_c \left[\prod_{j=1}^p P(x_j|A_c) \cdot P(A_c) \right]$$

se busca la clase A_c que maximiza la probabilidad posterior.

• SGD Classifier

Es un clasificador supervisado que entrena un modelo lineal (como regresión logística o SVM) usando descenso de gradiente estocástico.

Modelo: $f(X) = W^T X + b$

- $X \in \mathbb{R}^p$: vector de características
- $W \in \mathbb{R}^p$: pesos del modelo
- $b \in \mathbb{R}$: sesgo (bias)

Predicción: depende del signo de $f(X)$:

- Si $f(X) > 0$: predice la clase 1
- Si $f(X) < 0$: predice la clase 0

El modelo lineal traza una frontera de decisión que es un hiperplano que separa las clases.

Problema de Optimización: El SGD classifier intenta minimizar la función de pérdida promedio sobre todos los datos:

$$\min_{W, b} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda R(W)$$

Norma

Donde: • $L(y_i, f(x_i))$: función de pérdida

• y_i : etiqueta real de la muestra i

• x_i : vector de características de la muestra i

• $f(x_i)$: salida del modelo ($W^T x_i + b$)

• $R(W)$: término de regularización (ej. $L_2 = \|W\|^2$, $L_1 = \|W\|$)

• λ : fuerza de regularización

¿cómo se resuelve?: En lugar de calcular el gradiente sobre todos los datos (como el descenso de gradiente clásico), SGD hace lo siguiente:

1. Toma una muestra aleatoria (x_i, y_i)

2. calcula el gradiente de la pérdida con esa muestra:

$$\nabla W_i = \partial L_i / \partial W \quad \nabla b_i = \partial L_i / \partial b$$

3. se actualiza W y b .

$$W \leftarrow W - \eta \cdot \nabla W_i, \quad b \leftarrow b - \eta \cdot \nabla b_i$$

Donde: • η es la tasa de aprendizaje

• L_i es la pérdida en la muestra i

• Regresión logística

Es un modelo lineal de clasificación binaria, diseñado para predecir la probabilidad de que una muestra pertenezca a una clase.

Modelo: Dado un vector de características $x \in \mathbb{R}^p$, se quiere predecir:

$$P(y=1|x) = \sigma(W^T x + b)$$

Donde: • $W \in \mathbb{R}^p$: vector de pesos

• $b \in \mathbb{R}$: sesgo (bias)

• $\sigma(z) = \frac{1}{1+e^{-z}}$: función sigmoide o logística

esto da una salida entre 0 y 1, interpretada como una probabilidad

Función de pérdida: log-loss (cross-entropy)

La función de pérdida para una muestra (x_i, y_i) (con $g_i = \sigma(W^T x_i + b)$) es:

$$L(x_i, g_i) = -y_i \log(g_i) - (1-y_i) \log(1-g_i)$$

Esto penaliza fuertemente las predicciones que están lejos de la etiqueta verdadera

Problema de optimización: para un conjunto de entrenamiento de n muestras:

$$\min_{w, b} \frac{1}{n} \sum_{i=1}^n [-y_i \log(y_i) - (1-y_i) \log(1-y_i)] + \lambda \|W\|^2$$

- El primer término es la pérdida promedio (log-loss)
- El segundo término es la regularización L_2 , que controla el sobreajuste.

Derivadas (gradientes): para una sola muestra, con $z = W^T x + b$

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad L(w, b) = -y \log(\sigma(z)) - (1-y) \log(1-\sigma(z))$$

Gradientes: • Respecto a W : $\nabla_W L = (\sigma(z) - y) \cdot x$

• Respecto a b : $\nabla_b L = \sigma(z) - y$

se aplica la regla de actualización por descenso de gradiente, explicada en SGD classifier)

Linear Discriminant Analysis (LDA)

LDA es un clasificador supervisado lineal que:

- Asume que los datos de cada clase siguen una distribución gaussiana con la misma matriz de covarianza
- Busca proyectar los datos en una dirección que maximice la separación entre clases (discriminación)
- Se basa en principios probabilísticos y genera una proyección lineal.

Supuestos clave: 1) cada clase C_k genera datos $x \in \mathbb{R}^d$ con una distribución normal multivariada: $x \sim N(\mu_k, \Sigma)$, donde μ_k es la media de la clase y Σ es la misma matriz de covarianza para todas las clases.

2) La matriz de covarianza es igual para todas las clases: $\Sigma_1 = \Sigma_2 = \dots = \Sigma$

3) se conoce o se estima la probabilidad a priori de cada clase $p(C_k)$

Modelo de LDA: LDA predice la clase como:

$$\hat{g} = \arg \max_k \tilde{d}_k(x)$$

Donde $\tilde{d}_k(x)$ es la función discriminante lineal para la clase C_k :

$$\tilde{d}_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log p(C_k)$$

- La función $\tilde{d}_k(x)$ es lineal en x , por eso LDA es un modelo lineal.

• **KNeighbors Classifier:** Es un clasificador supervisado no-paramétrico.

Esto significa que no aprende una función o modelo durante el entrenamiento, sino que guarda todo el conjunto de entrenamiento.

Cuando llega un nuevo punto x , el modelo:

- 1-) Calcula la distancia entre x y todos los puntos del conjunto de entrenamiento.
- 2-) Selecciona los K puntos más cercanos (los vecinos).
- 3-) Asigna la clase más común entre esos vecinos a x .

El modelo de Clasificación de KNN es implícito, pero puede expresarse de la siguiente manera; Dado un conjunto de entrenamiento:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

y una nueva muestra x , la predicción es:

$$\hat{y} = \text{modo} \{y_i | x_i \in N_K(x)\}$$

donde:

- $N_K(x) \rightarrow$ Conjunto de los K vecinos más cercanos a x según alguna métrica (como distancia euclídea).
- $\text{modo} \rightarrow$ Clase más frecuente entre los vecinos.

KNN no optimiza una función de costo explícita, pero puede entenderse como un método que minimiza el error de clasificación localmente.

$$\hat{y}(x) = \arg \max_{c \in \text{clases}} \sum_{x_i \in N_K(x)} 1(y_i = c)$$

donde:

- $1(y_i = c)$ es una función indicadora que vale 1 si el vecino x_i tiene clase c , y 0 si no.

- $\hat{y}(x)$ es la clase que KNN va a predecir para el punto x .

- $c \in \text{clases}$: Se evalúa cada clase posible (ej. 0, 1, perro, gato, ...)

0 SVC: (Support Vector Classifier)

Es un clasificador Supervisado que busca encontrar un hiperplano lineal que separe dos clases con el máximo margen posible.

Es una versión lineal del SVM (Support Vector Machine), pero entrenado en un enfoque más eficiente.

El modelo lineal es:

$$f(x) = W^T x + b$$

La predicción se hace como:

$$\hat{y} = \text{Sign}(W^T x + b)$$

donde:

- W : Vector de pesos (dirección de hiperplano)

- b : Sesgo (desplazamiento del hiperplano)

- x : Vector de Características

Cabe resaltar que el margen es la distancia entre el hiperplano y los puntos más cercanos de cada clase, llamados Vectores de Soporte.

- Problema de Optimización
(Caso separable)

El problema es:

$$\min_{W, b} \frac{1}{2} \|W\|^2$$

$$\text{s.t. } y_i (W^T x_i + b) \geq 1, \text{ para todo } i$$

donde:

- $W^T x_i + b$: Predicción del modelo

- y_i : Salida real

Si $y_i (W^T x_i + b) \geq 1$, significa que el punto está bien clasificado y suficientemente lejos del hiperplano (margen).

Maximizar el margen equivale a minimizar $\|W\|^2$.

- Problema de Optimización
(Con datos no separables)

El problema es:

$$\min_{W, b, \xi} \frac{1}{2} \|W\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y_i (W^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

donde:

- ξ_i : Cuánto se permite que el punto se equivoque.

- C : hiperparámetro que controla el trade-off entre margen y errores de clasificación.

• Random Forest Classifier

El random Forest es un ensamble de árboles de decisión que combina múltiples árboles para mejorar la generalización y robustez del clasificador. Cada árbol se entrena de manera independiente usando:

- Bootstrap (muestras aleatorias con reemplazo)
- selección aleatoria de características en cada división.

El modelo consiste en un conjunto de T árboles de decisión:

$$\text{RandomForestClassifier} = \{h_1(x), h_2(x), \dots, h_T(x)\}$$

Cada $h_t(x)$ es un árbol que predice una clase para una muestra x . La predicción final se hace por votación mayoritaria (para clasificación):

$$\hat{y} = \text{mode}(\{h_t(x)\}_{t=1}^T)$$

Problema de optimización

Aunque no optimiza una función global explícita, cada árbol individual sí resuelve un problema de optimización greedy local al construir su estructura. Para cada nodo del árbol, se busca la mejor división de los datos que maximice la ganancia de información.

Problema de optimización por nodo

En cada nodo de un árbol, el algoritmo elige la característica j y el umbral θ que maximizan alguna medida de impureza:

- Gini impurity: $G(S) = 1 - \sum_{k=1}^K p_k^2$

- Entropy: $H(S) = - \sum_{k=1}^K p_k \log(p_k)$, p_k = Proporción de clases k en el nodo

Paso de optimización local

Para cada división candidata (j, θ) , se evalúa:

$$\text{Gain}(j, \theta) = \text{Impureza}(S) - \left(\frac{|S_L|}{|S|} \text{Impureza}(S_L) + \frac{|S_R|}{|S|} \text{Impureza}(S_R) \right)$$

se selecciona (j^*, θ^*) que maximiza esa ganancia

• Gaussian Process Classifier

Es un modelo bayesiano no paramétrico que asume que la función de decisión subyacente es una realización de un proceso gaussiano. Esto lo hace especialmente útil cuando se requiere una estimación bien calibrada de la incertidumbre.

Problema de optimización

Inferir la distribución posterior del vector latente $f = [f(x_1), \dots, f(x_N)]^T$, y predecir en nuevos puntos.

Paso 1: Modelo de predicción

$$p(y|f) = \prod_{i=1}^N \Phi(y_i; f_i) \quad (\text{esto asume } y_i \in \{-1, +1\} \text{ para simplificar})$$

Paso 2: Priori gaussiana sobre f

$f \sim N(0, K)$ donde $K \in \mathbb{R}^{N \times N}$ es la matriz de covarianza del kernel $k(x_i, x_j)$

Como la función posterior: $p(f|x, y) \propto p(y|f) \cdot N(f|0, K)$ no es gaussiana (por la función sigmoide), no se puede calcular exactamente.

Para esto se aproxima por Laplace, así:

Se aproxima el posterior $p(f|x, y)$ por una Gaussiana ajustada alrededor del modo a posteriori (MAP)

Como tal, el problema de optimización

$$f^* = \arg \max_f \log p(y|f) - \frac{1}{2} f^T K^{-1} f$$

• Primer término: log-verosimilitud (Ejm: logit o probit)

• Segundo término: regularización por la priori gaussiana.

Este problema se resuelve usando métodos de optimización de segundo orden como Newton-Raphson o Laplace's method.

• Clasificadores basados en Deep Learning

(CNN (Convolutional Neural Network))

El objetivo es aprender una función de clasificación:

$$f_\theta: \mathbb{R}^{28 \times 28} \rightarrow \{0, 1, \dots, 9\}$$

Donde:

- $X_i \in \mathbb{R}^{16 \times 16}$: imagen en escala de grises de 16×16 píxeles
- $y_i \in \{0, 1, \dots, 9\}$: clase verdadera (etiqueta) de la imagen
- $f_\theta(\cdot)$: red neuronal convolucional parametrizada por θ , que produce logits (puntuajes sin normalizar) para cada clase.

la salida de la red se convierte en probabilidades con softmax:

$$\hat{y}_i = \text{softmax}(f_\theta(x_i)) \in [0, 1]^{10}$$

Problema de optimización

El entrenamiento busca minimizar la función de pérdida de entropía cruzada (cross-entropy) sobre el conjunto de entrenamiento:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta) = - \sum_{i=1}^N \log \hat{y}_{i, y_i} = - \sum_{i=1}^N \log \left(\frac{e^{z_{i, y_i}}}{\sum_{c=1}^{10} e^{z_{i, c}}} \right)$$

Donde:

- $\hat{y}_{i, y_i}$: Probabilidad predicha para la clase verdadera.
- $z_{i, c}$: logit (output antes del softmax) para clase c .
- θ : todos los parámetros (pesos y sesgos) del modelo.
- N : # de ejemplos en el conjunto de entrenamiento

Evaluación del clasificador

Después del entrenamiento, el modelo es evaluado en el conjunto de prueba mediante:

- Exactitud (Accuracy): $\text{Acc} = \frac{\text{\# de predicciones correctas}}{N_{\text{test}}}$
- F₁-score macro: media del F₁ por clase
- Matriz de confusión: distribución de errores por clase
- Curva ROC (macro-average AUC): desempeño de clasificación multiclase en términos de sensibilidad vs especificidad.